HR Expert — Automated Candidate Evaluation System (Execution Document)

Purpose

Automate the end-to-end process of resume parsing, candidate evaluation, and communication using AI (OpenAI GPT-4 Turbo) within n8n.
This workflow handles everything from candidate submission to evaluation, data logging, and automatic email notifications.

Main Sources

- **Candidate resume uploads** (via n8n Form Trigger) — PDF format.
- **Google Sheets Candidate Database** — stores structured evaluation results.
- **Manual job profile input** — defined in the *Profile Wanted* node by HR.

Models

- **Primary Model:** OpenAI GPT-4 Turbo — used for parsing, summarization, and AI evaluation.
- **Optional Fallback:** Gemini 1.5 Flash (via Lovable) — for faster summarization tasks.

Primary Tools

- **n8n** — workflow automation
- **OpenAI** — LLM for parsing, summarization, and evaluation
- **Google Drive** — for CV uploads

- **Google Sheets** — for logging results
- **Gmail / SMTP** — for sending selection or rejection emails
- **ngrok** — for local testing and webhook exposure

High-Level Flow (One Line)

Candidate submits form → Upload CV → Extract text → AI parses data → Merge and summarize → Compare with job profile → AI evaluation → Conditional email → Log to Google Sheet.

Workflow Snapshot (Nodes & Roles)

1. Form Trigger (Candidate Submission)

Node: *Form Trigger*

Captures:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "file": "resume.pdf"
}
```

- Responds immediately with acknowledgment (“Application received”).
- Passes data (binary + JSON) to next node.

2. Upload CV (Google Drive Node)

- Uploads candidate resume to a dedicated Drive folder.
- Returns the file ID and public link for reference.

Example output:

```
{
```

```
  "fileId": "1a2b3c",
  "fileUrl": "https://drive.google.com/file/d/1a2b3c/view"
}
```


3. Extract from File (PDF Extractor Node)

- Reads the uploaded PDF.
- Extracts visible text and basic metadata.

Outputs structured text data:


```
 {
  "text": "John Doe, Email: john@example.com, Skills: Python, Java...",
  "metadata": { "pages": 2, "createdAt": "2025-10-31" }
}
```


4. Personal Data Model (OpenAI Node)

Prompt Example:

You are a resume parser. Extract and return only structured JSON:


```
{
  "name": "",
  "email": "",
  "telephone": "",
  "city": "",
  "birthdate": ""
}
```


Output Example:


```
{
  "name": "John Doe",
  "email": "john@example.com",
  "telephone": "+91 9876543210",
  "city": "Hyderabad, India"
}
```

5. Qualifications Model (OpenAI Node)

Prompt Example:

Extract the candidate's education, work experience, and technical skills.
Return strict JSON:

```
{
  "education": "",
  "experience": "",
  "skills": []
}
```

Output Example:

```
{
  "education": "B.Tech in Computer Science",
  "experience": "2 years as Frontend Developer",
  "skills": ["React", "JavaScript", "HTML", "CSS"]
}
```

6. Merge Node

Combines **Personal Data** and **Qualifications** into a single JSON object:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "telephone": "+91 9876543210",
  "education": "B.Tech in CSE",
  "skills": ["React", "JavaScript", "HTML", "CSS"],
  "experience": "2 years"
}
```

7. Summarization Chain (OpenAI Node)

Creates a short professional summary:

> “John Doe is a frontend developer with 2 years of experience, skilled in React, JavaScript, and modern web technologies.”

8. Profile Wanted (Manual Input Node)

Stores the HR-defined job profile:

```
{
  "role": "Frontend Developer",
  "required_skills": ["React", "JavaScript", "CSS"],
  "minimum_experience": 2
}
```

9. HR Expert Evaluation (AI Node)

Prompt Example:

Compare the candidate profile with the given job description.
Output JSON only:

```
{
  "vote": "",
  "consideration": ""
}
```

Vote should be between 1 and 10, where 10 = highly suitable.

Output Example:

```
{
  "vote": "9",
  "consideration": "Excellent technical match and relevant experience."
}
```

10. Structured Output Parser

Converts AI output to clean JSON:

```
{
  "vote": 9,
```

```
  "consideration": "Excellent technical match and relevant experience."
}
```

11. If Node (Conditional Logic)

Checks:

- If `vote >= 8` → Candidate selected (trigger success email).
- If `vote < 5` → Candidate rejected (trigger polite rejection email).

12. Email Nodes

Selected Email (Gmail Node):

Subject: Congratulations — You've Been Shortlisted!
Body:
We are pleased to inform you that you have been selected for the interview round.
Our HR team will reach out soon with further details.

Rejection Email (Gmail Node):

Subject: Regarding Your Application
Body:
Thank you for applying. After reviewing your profile, we found it doesn't meet our current requirements.
We encourage you to reapply for future opportunities.

13. Google Sheets (Append Row Node)

Appends candidate data to the HR Evaluation Sheet:

Date	Name	Email	City	Skills	Vote	Consideration	Decision
31/10/2025	John Doe	john@example.com	Hyderabad	React, JS	9	Excellent match	Selected

File / Workflow Names

- `HRExpertWorkflow.json` — n8n export

Key Node Configurations

Form Trigger

Method: POST
Path: `/webhook/candidate-submit`
Response: { "success": true, "message": "Application received" }

Evaluation Prompt

You are an HR evaluation assistant. Your task is to assess how well a candidate fits the company's desired job profile.

Profile Sought
{{ $json.profile_wanted }}

Candidate Summary
{{ $('Summarization Chain').item.json.response.text }}

Evaluation Criteria
1. Relevance of technical skills and tools used.
2. Alignment of experience and education with the role.
3. Soft skills and extracurricular activities relevant to the company culture.
4. Problem-solving, initiative, and project quality.

Instructions

Based on the above profile and candidate summary, evaluate the candidate on a scale of **1 to 10**, where:
- **10** → The candidate is an exceptional match; must be shortlisted immediately.
- **7–9** → Strong candidate; consider for next interview round.
- **4–6** → Average match; consider only if limited applicants.
- **1–3** → Not a suitable match for the role.
Output Format (strict JSON)
Return only a JSON object with the following structure:
{
  "vote": <number between 1 and 10>,
  "consideration": "<brief paragraph explaining why the candidate received this score and whether they should be considered>"
}

Make sure your reasoning in *consideration* reflects skill alignment, experience relevance, and overall suitability for the profile.

If Node Expression

Convert vote to number before comparison:

{{ Number($json.vote) >= 8 }}

Google Sheets Configuration

Sheet Name: Candidate Evaluations
Columns:

- Date
- Name
- Email
- City
- Skills
- Summary
- Vote
- Consideration
- Decision

Error Handling

- PDF extraction failure → Respond with message “Resume unreadable. Please re-upload.”
- AI timeout → Retry node (2 attempts).
- Email send failure → Log to Google Sheet + notify admin.

Testing Checklist

1. Submit test form → confirm webhook execution.
2. Check extracted text in “Extract from File” node.
3. Validate AI model outputs for personal and qualification data.
4. Confirm merged output structure.
5. Verify evaluation node returns vote and consideration.
6. Test “If” condition and email logic.
7. Ensure Google Sheet logs entries correctly.

Example Payloads

Webhook Input


```
{
  "name": "John Doe",
  "email": "john@example.com",
  "file": "resume.pdf"
}
```


AI Evaluation Output


```
{
  "vote": "9",
  "consideration": "Strong skill alignment and relevant experience."
}
```

Deliverables

1. `HR_Expert_Workflow.json` — ready-to-import n8n workflow.
2. AI model prompts (`personal_data_prompt.txt`, `evaluation_prompt.txt`, etc.).
3. Gmail HTML templates for selection/rejection.
4. Sample Google Sheet with structure and headers.
5. Step-by-step deployment guide (ngrok → Gmail → Sheets).